Spark Fuse ComfyUI: Complete Setup and User Guide

This guide takes you from a bare machine to rendering ComfyUI workflows on a Spark Fuse cloud GPU. It assumes no prior knowledge. You stage your models once, then render either from the command line or with a one-click button inside ComfyUI, with warm runs finishing in about a minute.

You do not need Docker. The container image runs on Spark Fuse's GPUs, not on your machine.

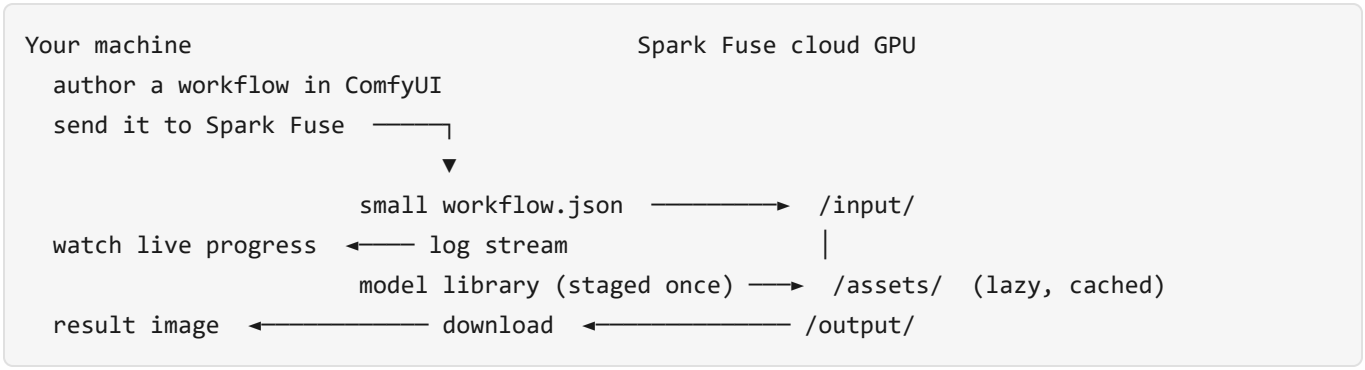
Contents

1. How it works

Heavy ComfyUI workflows (Flux and similar) run on rented cloud GPUs instead of your own machine. The parts are:

- **The image**: a Docker image that runs ComfyUI headlessly, published at `ghcr.io/vfxguru/spark-fuse-comfyui`. Spark Fuse pulls it for you; you never build or run it.
- **ShareSync**: Spark Fuse's cloud storage, where your models, workflow, and results live. The Spark desktop app shows it as an ordinary folder.
- **The messenger**: a command line tool that submits jobs (Path A).
- **The ComfyUI node**: an extension that submits the current workflow with a button (Path B).

Your local ComfyUI and the cloud ComfyUI never talk directly. Your machine is the authoring canvas; everything moves as files on ShareSync.



The model library is staged on ShareSync once and mounted read-only and lazily at `/assets`, cached on the compute node between jobs. Only the small `workflow.json` is sent each render. With cached-image affinity, a warm run skips both the image pull and the model copy, which is why repeat renders are fast.

2. Before you start

Accounts

- A **Spark Fuse account** with API access: a host URL, an email, and a password. Sign up at sparkcloud.studio.

Prerequisites by path

You only need the prerequisites for the path you choose in section 5.

Common (both paths): - Windows 10/11 or macOS. - The Spark desktop app (section 3).

Path A, the command line, also needs: - **Python 3.12 or newer**: python.org/downloads. During install on Windows, tick "Add Python to PATH". - **Git**: git-scm.com/downloads. - **uv** (a fast Python tool). Install it once: `powershell powershell -c "irm https://astral.sh/uv/install.ps1 | iex"` (macOS or Linux: `curl -LsSf https://astral.sh/uv/install.sh | sh`.) Open a new terminal afterwards so `uv` is on your PATH.

Path B, the ComfyUI node, also needs: - A working **local ComfyUI** install. See ComfyUI. Note the folder where ComfyUI lives and which Python runs it. - **Git** (as above).

3. Install the Spark desktop app (ShareSync)

This app is how your models and workflows reach ShareSync.

1. Download the Spark desktop app from sparkcloud.studio/downloads for Windows or macOS.
2. Run the installer and launch the app.
3. Sign in with your Spark account (the same email and password).

4. The app mounts your ShareSync storage as a synced folder. On Windows it appears in File Explorer (for example under your user folder as "Spark ShareSync"); on macOS it appears in Finder. Open it and confirm you can see your space. Anything you put in this folder syncs to ShareSync and becomes available to your jobs.

If the folder does not appear, make sure the app is signed in and running, then check its settings for the sync location.

4. Stage your model library (once)

Spark Fuse reads your models from ShareSync, so you upload them once and reuse them on every job. Lay the library out to **mirror your local ComfyUI `models` folder**, including any subfolders (for example `diffusion_models/FLUX2/...`). A workflow refers to a model by its path, so if a model sits in a subfolder locally it must sit in the same subfolder here, or the cloud will not find it. Windows path separators are handled for you: the ComfyUI node converts a Windows `\` to the Linux `/` automatically.

1. In the Spark ShareSync folder, create a folder for your models, for example `comfy-models`.
2. Inside it, create the usual ComfyUI subfolders and copy your model files in: `comfy-models/ ├── diffusion_models/ e.g. flux-2-klein-9b-fp8.safetensors ├── text_encoders/ e.g. the text encoder your workflow loads ├── vae/ ├── checkpoints/ ├── loras/ └── controlnet/` Use only the subfolders your workflows need.
3. Let the desktop app finish uploading. You can confirm in the ShareSync web interface that the folder shows its full size. This upload runs at your connection's upload speed and only happens once.

The path you pass to a job is the path as it appears in ShareSync. A folder you see as `comfy-models` at the root of your Personal space is the path `/comfy-models/`.

5. Choose your path

- **Path A, the command line**, is best for scripting and batch work. Continue at section 6.
- **Path B, the ComfyUI node**, is the friendliest: a button inside ComfyUI. Continue at section 7.

Both use the same staged models and produce the same result. You can set up both.

6. Path A: render from the command line

6.1 Install the messenger (once)

```
git clone https://github.com/VFXGuru/spark-fuse-messenger
cd spark-fuse-messenger
uv sync
```

Copy `.env.example` to `.env` and fill in your details:

```
SPARK_HOST=https://api.prod.aapse1.sparkcloud.studio
SPARK_EMAIL=you@yourcompany.com
SPARK_PASSWORD=your-password
```

Activate the environment and confirm it works:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
.\.venv\Scripts\Activate.ps1
spark-fuse login
```

(macOS or Linux: `source .venv/bin/activate`.) You repeat the activate step in each new terminal.

6.2 Author and export the workflow

Build your workflow in ComfyUI with a Save Image node at the end. Export it in **API format** (the workflow menu's **Export (API)** option, not an ordinary save). Save the file as `workflow.json`. The API format is required; an ordinary save is rejected.

6.3 Stage the workflow

In the Spark ShareSync folder, create a small folder, for example `my-job`, and copy only `workflow.json` into it. The models do not go here; they are served from `/assets`. Wait for it to sync.

6.4 Submit

```
spark-fuse submit --image ghcr.io/vfxguru/spark-fuse-comfyui:latest --command python3.13 --
command /runner/spark_fuse_run.py --instance-type g6.xlarge --input-path "/my-job/" --assets-
path "/comfy-models/" --env MODEL_BASE_DIR=/assets --image-affinity required
```

- `--input-path` is the small workflow folder; `--assets-path` is your model library; `MODEL_BASE_DIR=/assets` tells ComfyUI to read models from the assets mount.

- It prints a job ID. If you get an immediate HTTP 400 about the input path, the folder has not finished syncing yet; confirm it in the web interface and resubmit. No GPU is billed in that case.
- Tip: for the most reliable warm-start affinity, pin the image by digest (`...@sha256:<digest>`) instead of `:latest` .

6.5 Watch and download

```
spark-fuse logs <job-id>
spark-fuse download <job-id> .\results
```

Connect to the logs straight away (there is no replay). When the job succeeds, the image downloads into `.\results` .

7. Path B: render from inside ComfyUI (the node)

7.1 Install the node (once)

1. Clone it into your ComfyUI `custom_nodes` folder: `powershell cd <your ComfyUI folder>\custom_nodes git clone https://github.com/VFXGuru/spark-fuse-comfyui-node`
2. Install its dependency (the messenger client) into **the same Python that runs your ComfyUI**. Use the option that matches your install:

Standard install (a venv or system Python): `powershell pip install -r spark-fuse-comfyui-node\requirements.txt`

Portable or desktop ComfyUI (embedded Python): the portable build ships its own Python in a `python_embeded` folder, and its isolated build step cannot fetch the messenger's build backend, which fails with `Cannot import 'hatchling.build'` . Install the backend first and skip isolation. Open a terminal in the `python_embeded` folder and run: `powershell .\python.exe -m pip install hatchling .\python.exe -m pip install --no-build-isolation -r "..\ComfyUI\custom_nodes\spark-fuse-comfyui-node\requirements.txt"` Adjust the path to `requirements.txt` if your folder layout differs. Confirm it worked: `powershell .\python.exe -c "import spark_fuse; print('spark_fuse OK')"` 3. Restart ComfyUI.

7.2 Configure

1. Click the ⚡ **Spark Fuse** button (top right of the ComfyUI window).
2. Open **Credentials** and enter your host, email, and password. (Alternatively set `SPARK_HOST` , `SPARK_EMAIL` , and `SPARK_PASSWORD` in the environment.)
3. Set the **Assets ShareSync path** to your model library, for example `/comfy-models/` .
4. Pick a **GPU**; the hourly rate appears beside it.
5. Click **Save settings**.

7.3 Render

1. Build or open a workflow with a Save Image node at the end.
2. Click ⚡ **Spark Fuse**, then **Render on Spark Fuse**. (The button stays greyed until the GPU list has loaded.)
3. Watch progress in the panel, and leave ComfyUI running until the image returns. When the job finishes, the image appears in the panel and is saved into ComfyUI's output folder under the next sequential name, so repeated renders accumulate rather than overwrite.

If the button does not appear after restarting, open your browser's developer console and the ComfyUI log and check for errors, and confirm the dependency installed into the same Python as ComfyUI.

8. Choosing a GPU and seeing the cost

```
spark-fuse skus
spark-fuse estimate g6.xlarge
```

`skus` lists every GPU type with its memory. `estimate` shows the hourly rate. Pick the smallest GPU that fits your models to keep cost down, or a larger one for the fastest single render. As a guide, a model set of around 17 GB runs comfortably on any 24 GB card. The rate is per hour; your actual cost depends on how long the job runs, including the one-time cold start.

9. Troubleshooting

The runner reports a clear exit code, shown in the job summary:

Exit code	Meaning	What to do
0	Success	Download your output.
1	Workflow executed but ended in error	Read the ComfyUI error in the log; usually a bad node setting.
2	Bad input	`workflow.json` is missing, unreadable, or not API format; or a model path is wrong.
3	ComfyUI rejected the workflow	The log names the missing node type; that custom node is not in the image.
4	Job timed out	Increase the timeout, or check why the render stalled.
5	Completed but produced no files	Your workflow has no save node, so nothing was written to `/output`.
6	ComfyUI failed to start or died	Check the log near startup; report it if it persists.

Other common situations:

- **Submission fails with HTTP 400 about the input or assets path.** The folder has not finished syncing to ShareSync, or the path is misspelt. Confirm it in the ShareSync web interface, then resubmit.
- **Submission fails with `account_check_failed` or an estimate returns a pricing error.** That GPU type has no price configured. Choose a different instance type or ask Spark Cloud Studio to add the pricing row.
- **The workflow is rejected as the wrong format.** Re-export it using **Export (API)**, not an ordinary save.
- **A model is not found.** Check that the file sits under the correct subfolder of your library and that the name in the workflow matches exactly.
- **The first run is slow.** That is the expected cold start. Run it again and, with image affinity, the warm run is much faster.

10. How it works under the hood

- **Models load lazily and stay cached.** Your library is mounted read-only at `/assets`. Only the bytes a render reads are pulled, and they are cached on the compute node for the next job. Nothing is re-uploaded from your machine.
- **Image affinity** steers repeat runs onto a node that already cached the image, so the multi-gigabyte image pull is skipped. The first run on a freshly published image is a cold start; later runs are warm. `spark-fuse status <job-id>` reports whether a run hit the cache.
- **One workflow per job.** The whole graph runs in the cloud; your machine only authors it and collects the result.

License